🛡️ HACKER WOMEN COUNCIL — ETHICAL HACKING

Actividad 03 - SQL CHALLENGE

👤Karen Itzel Jaime Castillo

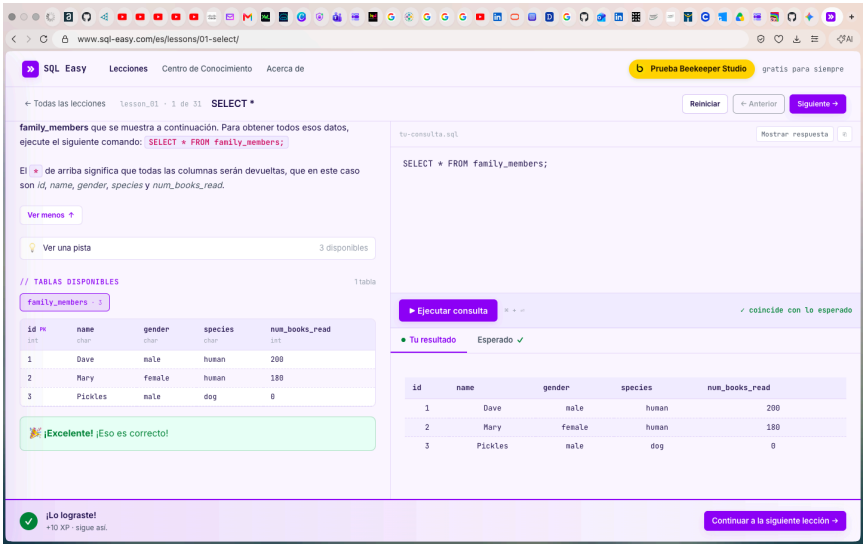
📄 Documentación Completa
🎯 Sitio utilizado: www.sql-easy-com
📅 Fecha: 27/10/2026

SQL EASY (DOCUMENTACIÓN DE LAS 31 CONSULTAS)

Completar y documentar los 31 ejercicios de SQL Easy en español.

Ejercicio 1: Seleccionar registros

- **Objetivo del ejercicio:** Obtener todos los registros y columnas de la tabla `family_members`.
- **Concepto SQL:** `SELECT *`
- **Consulta ejecutada:** `SELECT * FROM family_members;`
- **Resultado:**

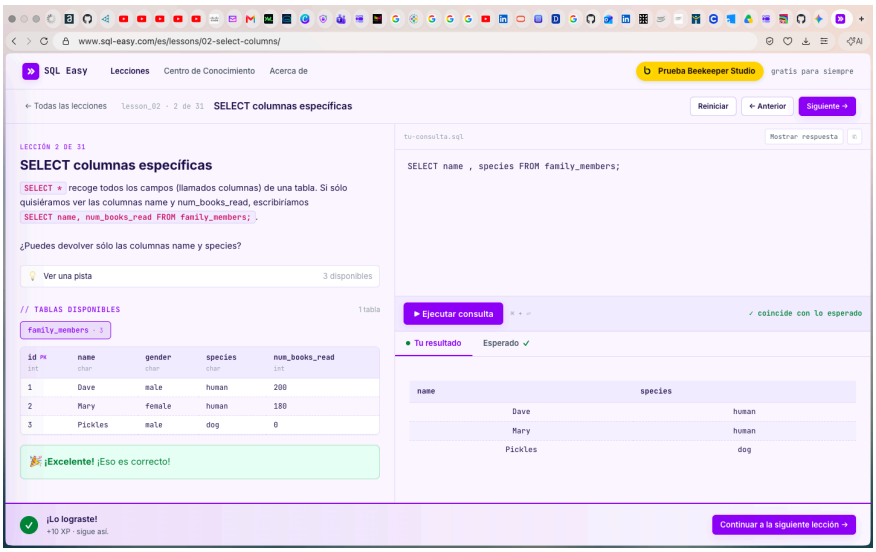


- **Interpretación:** La consulta devuelve una tabla completa con todos los datos y campos disponibles de cada uno de los miembros de la familia registrados.

- **Aprendizaje:** Comprendí cómo funciona el asterisco (*) como comodín para seleccionar la totalidad de la información contenida en una tabla sin filtrar campos específicos.
- **Aplicación en el mundo real:** En una organización, esta consulta se utilizaría para realizar una auditoría rápida o una extracción general de toda la información de una tabla de usuarios o empleados antes de aplicar filtros más específicos.

Ejercicio 2: Seleccionar columnas específicas

- **Objetivo del ejercicio:** Recuperar únicamente los campos de nombre y especie de la tabla `family_members`.
- **Concepto SQL:** Proyección de columnas específicas (`SELECT column1, column2`)
- **Consulta ejecutada:** `SELECT name, species FROM family_members;`
- **Resultado:**

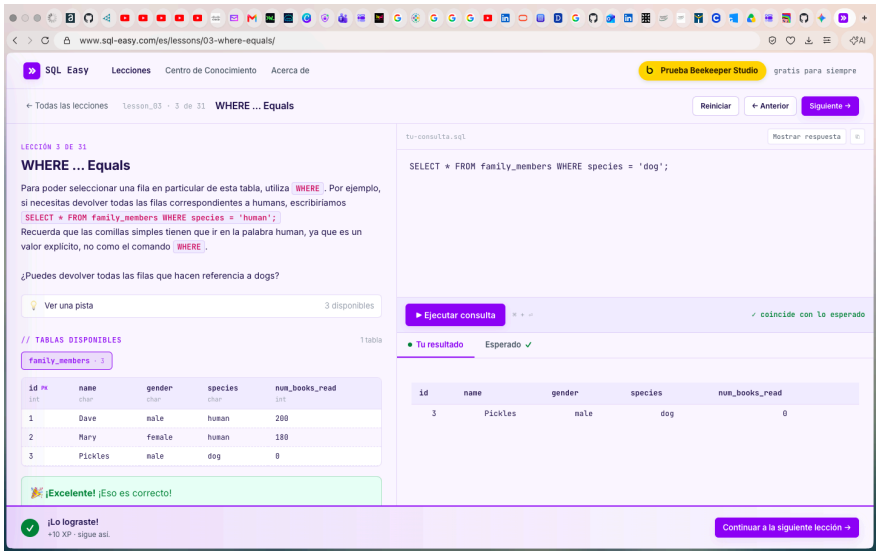


- **Interpretación:** La base de datos arroja una lista reducida que muestra únicamente los nombres y las especies de los miembros, omitiendo el resto de los atributos innecesarios.
- **Aprendizaje:** Aprendí la importancia de solicitar solo los datos necesarios en lugar de usar *, lo cual optimiza el rendimiento de la red y el procesamiento de las consultas.

- **Aplicación en el mundo real:** Ideal para generar reportes ejecutivos donde solo se requiere visualizar información clave (como nombre y correo de los clientes), protegiendo o ignorando datos sensibles o adicionales.

Ejercicio 3: Filtrar con WHERE (Igualdad)

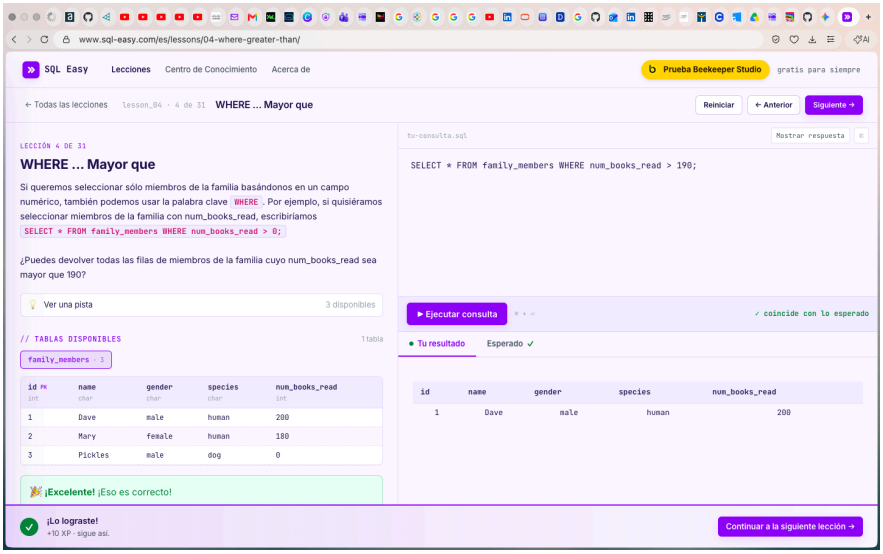
- **Objetivo del ejercicio:** Filtrar los registros de la tabla `family_members` para obtener únicamente aquellos cuya especie sea `'dog'`.
- **Concepto SQL:** Cláusula `WHERE`
- **Consulta ejecutada:** `SELECT * FROM family_members WHERE species = 'dog';`
- **Resultado:**



- **Interpretación:** Se filtran las filas de la tabla de modo que solo se muestran los registros de los miembros que cumplen estrictamente con la condición de ser perros.
- **Aprendizaje:** Comprendí cómo la cláusula `WHERE` actúa como un filtro condicional para restringir las filas devueltas basándose en un criterio de texto exacto.
- **Aplicación en el mundo real:** Se utiliza frecuentemente para buscar registros específicos en sistemas de gestión, como filtrar transacciones bancarias realizadas en una fecha o sucursal específica.

Ejercicio 4: Filtrar con Mayor que

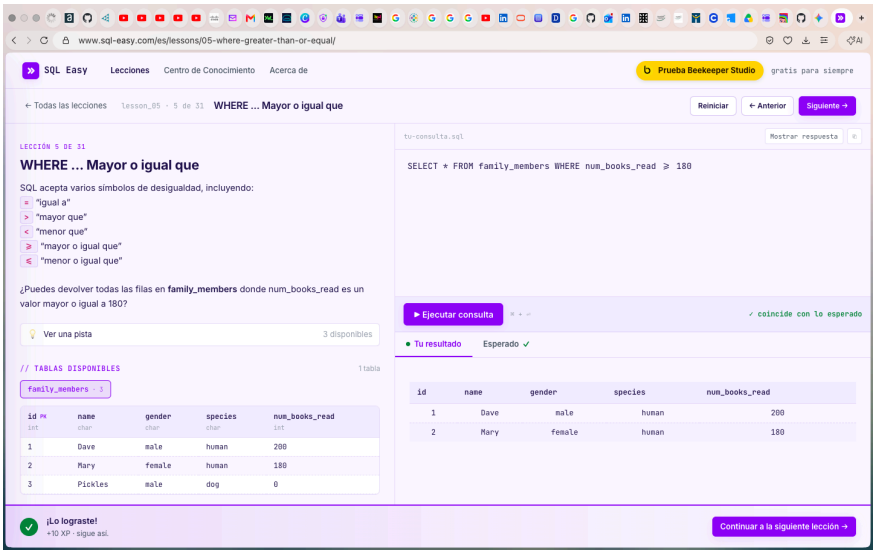
- **Objetivo del ejercicio:** Consultar los miembros de la familia que hayan leído una cantidad de libros estrictamente mayor a 190.
- **Concepto SQL:** Operador de comparación `>` (Mayor que)
- **Consulta ejecutada:** `SELECT * FROM family_members WHERE num_books_read > 190;`
- **Resultado:**



- **Interpretación:** Muestra únicamente los registros donde el valor del campo numérico `num_books_read` supera el umbral de 190, excluyendo a los que tengan 190 exactos o menos.
- **Aprendizaje:** Afiancé el uso de operadores lógicos y numéricos para realizar comparaciones de rangos en bases de datos relacionales.
- **Aplicación en el mundo real:** Muy útil en áreas de recursos humanos o analítica para filtrar empleados que superen metas de ventas, horas de capacitación o puntajes de rendimiento específicos.

Ejercicio 5: Filtrar con Mayor o igual que

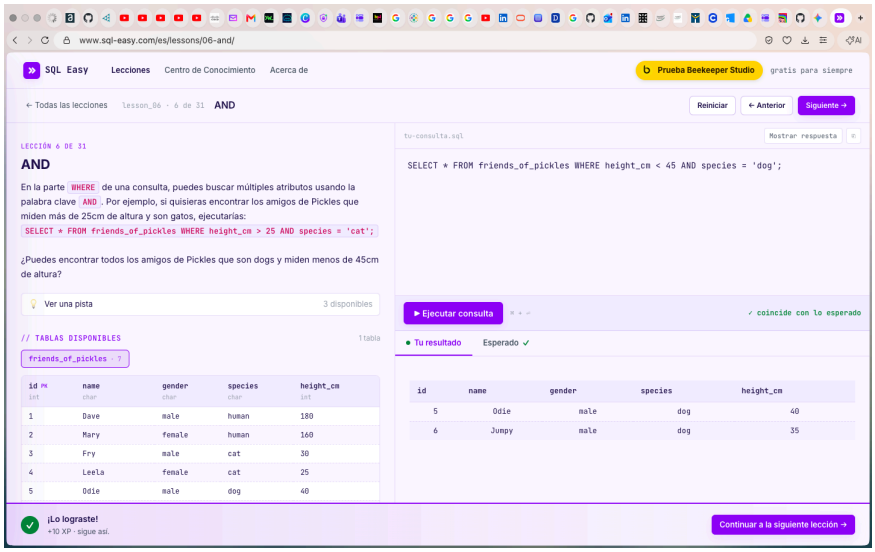
- **Objetivo del ejercicio:** Obtener los registros de los miembros que tengan un acumulado de 180 o más libros leídos.
- **Concepto SQL:** Operador de comparación `>=` (Mayor o igual que)
- **Consulta ejecutada:** `SELECT * FROM family_members WHERE num_books_read >= 180;`
- **Resultado:**



- **Interpretación:** La consulta devuelve todos los perfiles que cumplen con la condición inclusiva de tener 180 libros leídos en adelante.
- **Aprendizaje:** Entendí la diferencia de aplicar operadores inclusivos (`>=`) frente a estrictos (`>`) para la delimitación de rangos numéricos.
- **Aplicación en el mundo real:** Útil para filtrar inventarios con stock igual o menor al mínimo permitido, o listar usuarios elegibles para bonificaciones a partir de un puntaje mínimo.

Ejercicio 6: Condición múltiple con AND

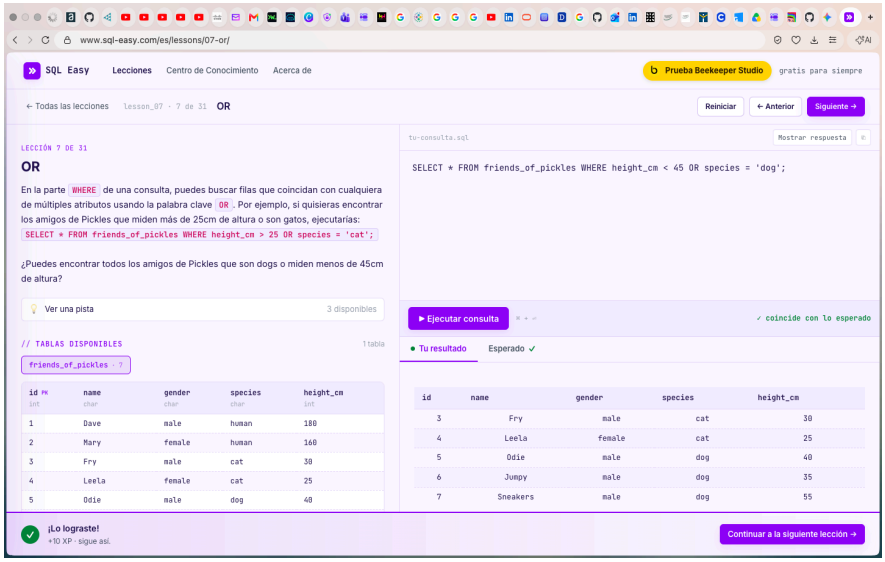
- **Objetivo del ejercicio:** Filtrar de la tabla `friends_of_pickles` aquellos amigos que midan menos de 45 cm y cuya especie sea `'dog'`.
- **Concepto SQL:** Operador lógico `AND`
- **Consulta ejecutada:** `SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';`
- **Resultado:**



- **Interpretación:** La base de datos filtra los registros exigiendo que se cumplan ambas condiciones de manera simultánea para que una fila sea mostrada.
- **Aprendizaje:** Comprendí cómo combinar múltiples criterios de búsqueda estrictos utilizando el operador `AND` para acotar resultados muy específicos.
- **Aplicación en el mundo real:** Aplicado en filtros de comercio electrónico, por ejemplo, para buscar productos que sean de una marca específica y que además estén en un rango de precio determinado.

Ejercicio 7: Condición alternativa con OR

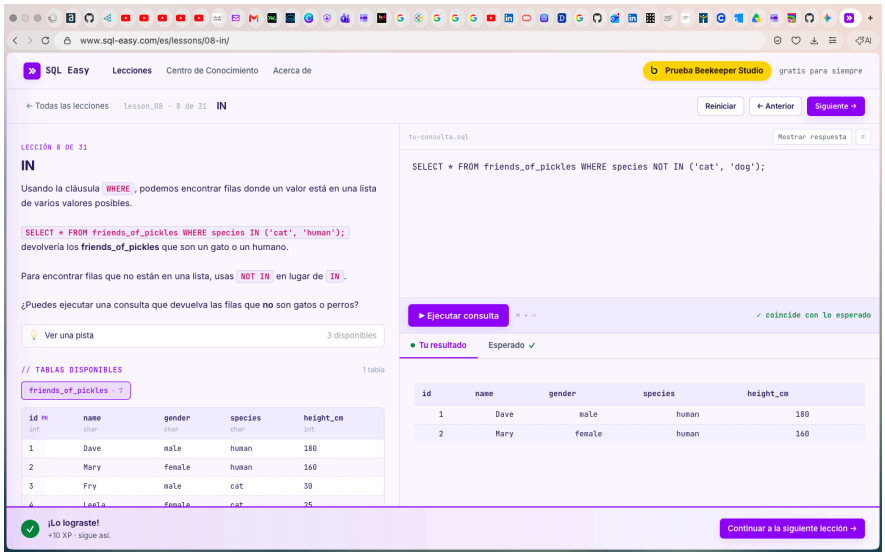
- **Objetivo del ejercicio:** Obtener los registros de `friends_of_pickles` donde la altura sea menor a 45 cm o la especie sea `'dog'`.
- **Concepto SQL:** Operador lógico `OR`
- **Consulta ejecutada:** `SELECT * FROM friends_of_pickles WHERE height_cm < 45 OR species = 'dog';`
- **Resultado:**



- **Interpretación:** Devuelve registros si se cumple al menos una de las dos condiciones; es decir, incluirá perros altos y también animales de otras especies que midan menos de 45 cm.
- **Aprendizaje:** Diferencié el comportamiento inclusivo y amplio del operador `OR` en contraste con la restricción estricta del `AND`.
- **Aplicación en el mundo real:** Útil en búsquedas abiertas donde el usuario acepta múltiples alternativas (ej. buscar vuelos que salgan un martes o un miércoles).

Ejercicio 8: Filtrado de exclusión con NOT IN

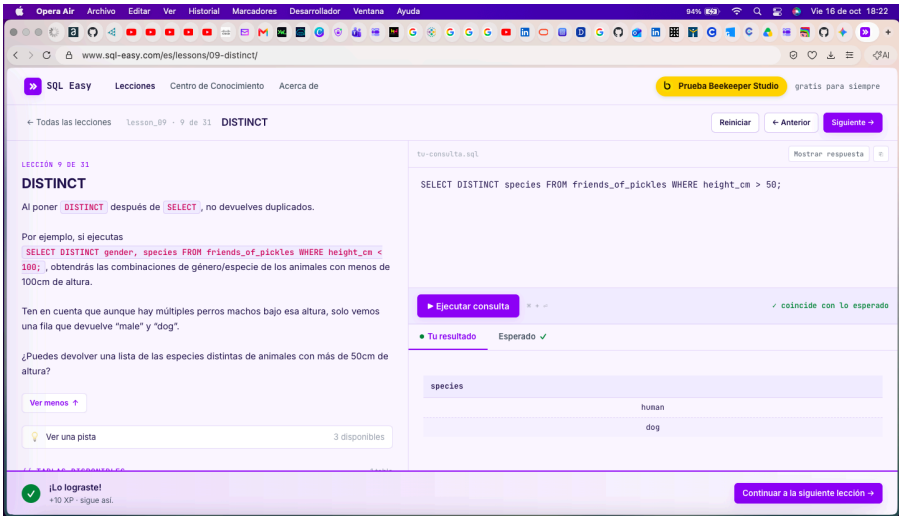
- **Objetivo del ejercicio:** Consultar los amigos de Pickles excluyendo aquellas filas cuya especie sea `'cat'` o `'dog'`.
- **Concepto SQL:** Operador de exclusión `NOT IN`
- **Consulta ejecutada:** `SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');`
- **Resultado:**



- **Interpretación:** Filtra la tabla descartando automáticamente todos los registros que pertenezcan a las categorías especificadas dentro del paréntesis.
- **Aprendizaje:** Aprendí a simplificar código SQL complejo evitando múltiples condiciones `AND species != ...` gracias al uso de listas con `NOT IN`.
- **Aplicación en el mundo real:** Esencial en ciberseguridad o administración de sistemas para excluir roles de usuario no autorizados o estados de cuentas bloqueados/suspendidos de un reporte general.

Ejercicio 9: Selección de valores únicos con DISTINCT

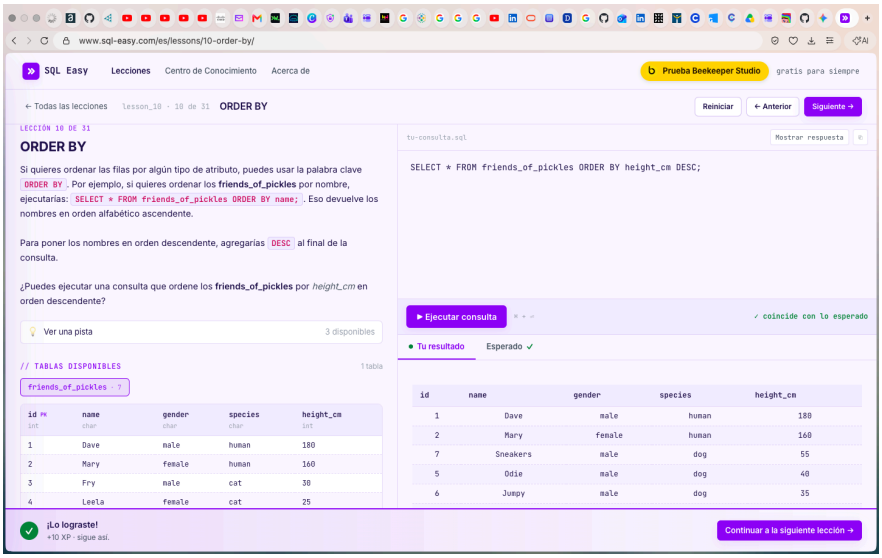
- **Objetivo del ejercicio:** Listar las especies únicas presentes en `friends_of_pickles` considerando únicamente a aquellos con una altura mayor a 50 cm.
- **Concepto SQL:** Cláusula `DISTINCT`
- **Consulta ejecutada:** `SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;`
- **Resultado:**



- **Interpretación:** Devuelve una lista resumida de especies sin repetir ninguna categoría, aplicando previamente el filtro de altura.
- **Aprendizaje:** Comprendí cómo eliminar registros duplicados en los resultados de una columna para obtener un panorama limpio de categorías o tipos de datos.
- **Aplicación en el mundo real:** Muy utilizado para generar listados de catálogos únicos (por ejemplo, ver todas las ciudades distintas desde donde se han registrado inicios de sesión en un sistema).

Ejercicio 10: Ordenamiento con ORDER BY

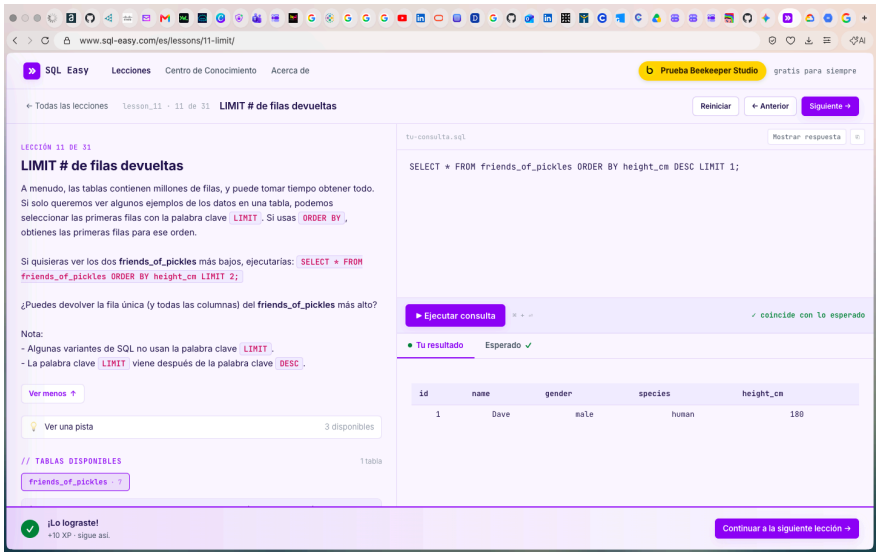
- **Objetivo del ejercicio:** Listar todos los registros de `friends_of_pickles` ordenados por su altura en forma descendente.
- **Concepto SQL:** Cláusula `ORDER BY ... DESC`
- **Consulta ejecutada:** `SELECT * FROM friends_of_pickles ORDER BY height_cm DESC;`
- **Resultado:**



- **Interpretación:** Organiza la tabla completa de mayor a menor basándose en la columna de altura (`height_cm`).
- **Aprendizaje:** Aprendí a controlar el orden en que se presentan los datos utilizando `ASC` (ascendente por defecto) o `DESC` (descendente).
- **Aplicación en el mundo real:** Fundamental en e-commerce y paneles analíticos para ordenar productos de mayor a menor precio, o mostrar las alertas de seguridad más críticas (de mayor riesgo a menor) primero.

Ejercicio 11: Limitar filas con LIMIT

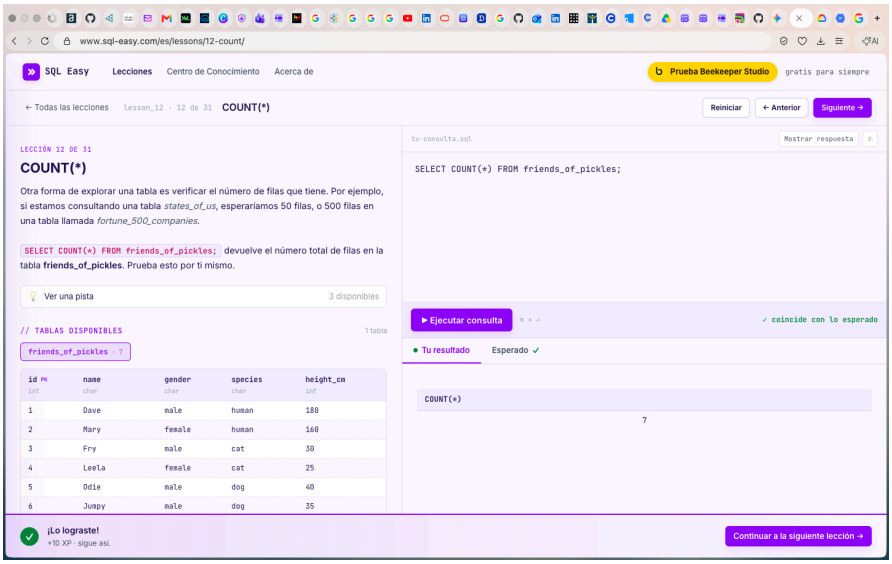
- **Objetivo del ejercicio:** Obtener únicamente el registro del amigo de Pickles que tenga la mayor altura.
- **Concepto SQL:** Cláusula `LIMIT` combinada con `ORDER BY DESC`
- **Consulta ejecutada:** `SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1;`
- **Resultado:**



- **Interpretación:** Ordena los datos de mayor a menor según la altura y restringe la salida para mostrar únicamente la primera fila (el valor máximo).
- **Aprendizaje:** Comprendí cómo optimizar y acotar la cantidad de resultados devueltos por una consulta, lo cual es muy útil para obtener los "Top N" registros.
- **Aplicación en el mundo real:** Se utiliza para mostrar el producto más vendido, el usuario con mayor puntuación en un ranking o la última transacción realizada en un sistema.

Ejercicio 12: Contar registros con COUNT

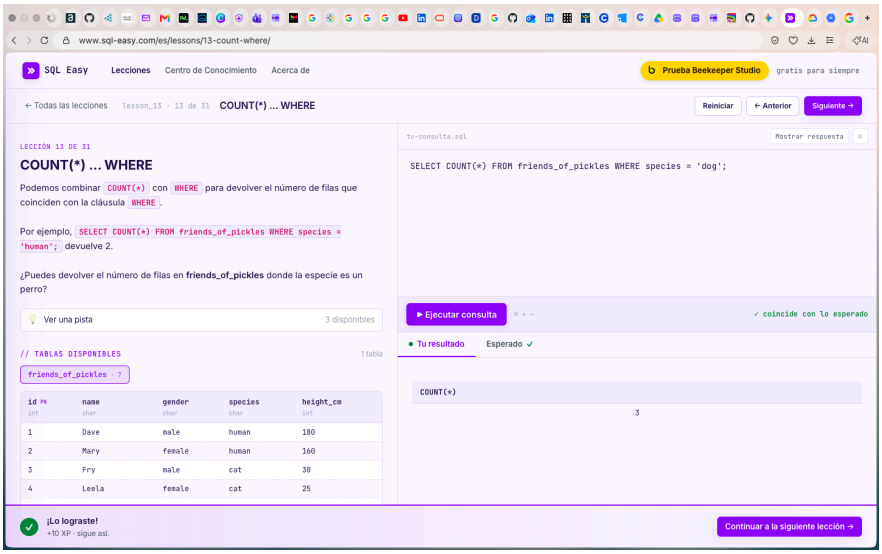
- **Objetivo del ejercicio:** Contar el número total de registros existentes en la tabla `friends_of_pickles`.
- **Concepto SQL:** Función de agregación `COUNT(*)`
- **Consulta ejecutada:** `SELECT COUNT(*) FROM friends_of_pickles;`
- **Resultado:**



- **Interpretación:** La base de datos procesa la tabla y devuelve un único valor numérico que representa el total de filas que contiene.
- **Aprendizaje:** Aprendí a utilizar funciones de agregación para resumir información cuantitativa en lugar de listar los registros individualmente.
- **Aplicación en el mundo real:** Fundamental para obtener métricas rápidas de negocio, como conocer la cantidad total de clientes registrados o productos en stock.

Ejercicio 13: Conteo con condición (COUNT con WHERE)

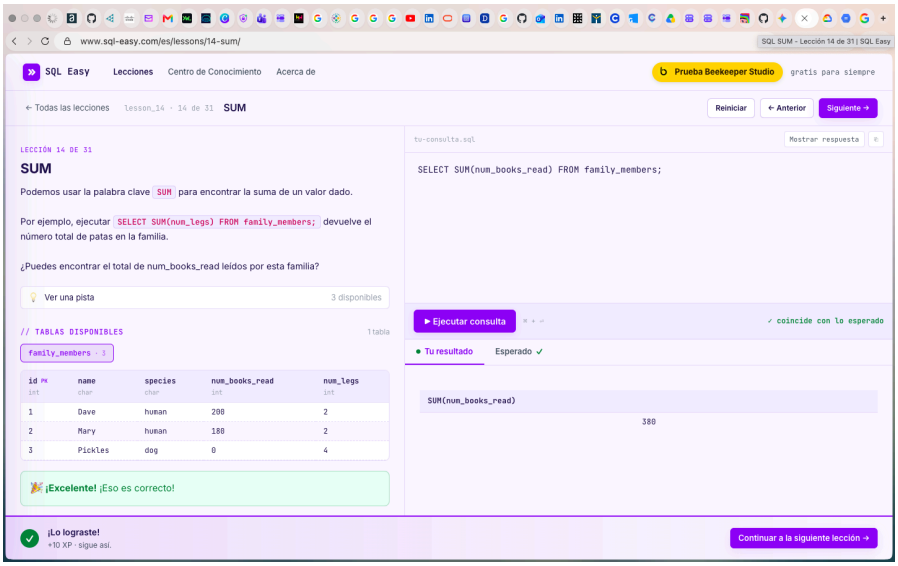
- **Objetivo del ejercicio:** Obtener la cantidad exacta de amigos de Pickles cuya especie sea `'dog'`.
- **Concepto SQL:** Función `COUNT(*)` combinada con la cláusula `WHERE`
- **Consulta ejecutada:** `SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';`
- **Resultado:**



- **Interpretación:** Filtra previamente las filas que cumplen con la condición de especie y posteriormente cuenta cuántas cumplen con dicho criterio.
- **Aprendizaje:** Afiancé la combinación de filtros lógicos con funciones de conteo para obtener estadísticas específicas y segmentadas.
- **Aplicación en el mundo real:** Muy útil para reportes operativos, por ejemplo, contar cuántos tickets de soporte técnico se encuentran abiertos o cuántos usuarios activos pertenecen a una región específica.

Ejercicio 14: Sumar valores con SUM

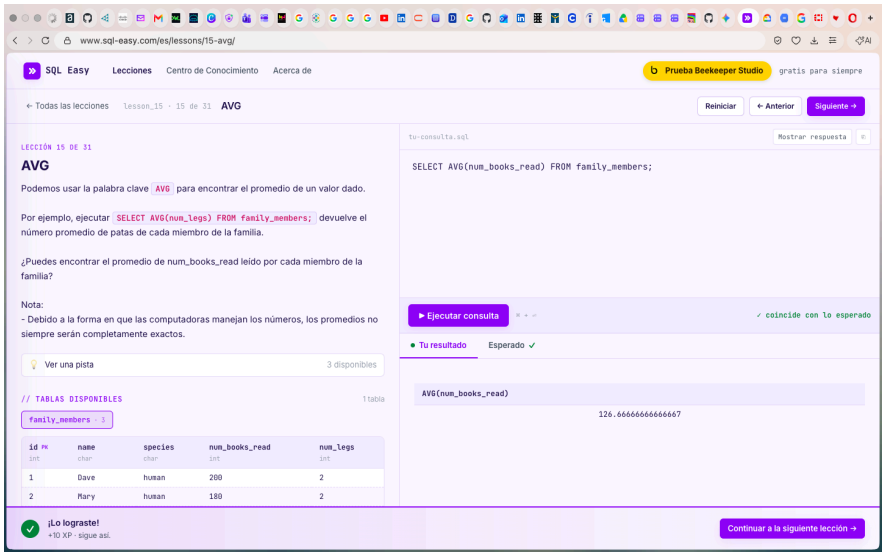
- **Objetivo del ejercicio:** Calcular la suma acumulada de todos los libros leídos por los integrantes de la familia.
- **Concepto SQL:** Función de agregación `SUM()`
- **Consulta ejecutada:** `SELECT SUM(num_books_read) FROM family_members;`
- **Resultado:**



- **Interpretación:** Suma los valores numéricos de la columna `num_books_read` de todas las filas y devuelve el total general acumulado.
- **Aprendizaje:** Comprendí cómo operar matemáticamente sobre columnas numéricas enteras para consolidar métricas globales.
- **Aplicación en el mundo real:** Esencial en áreas financieras y comerciales para calcular el total de ventas mensuales, costos operativos o sumas de inventarios.

Ejercicio 15: Calcular promedio con AVG

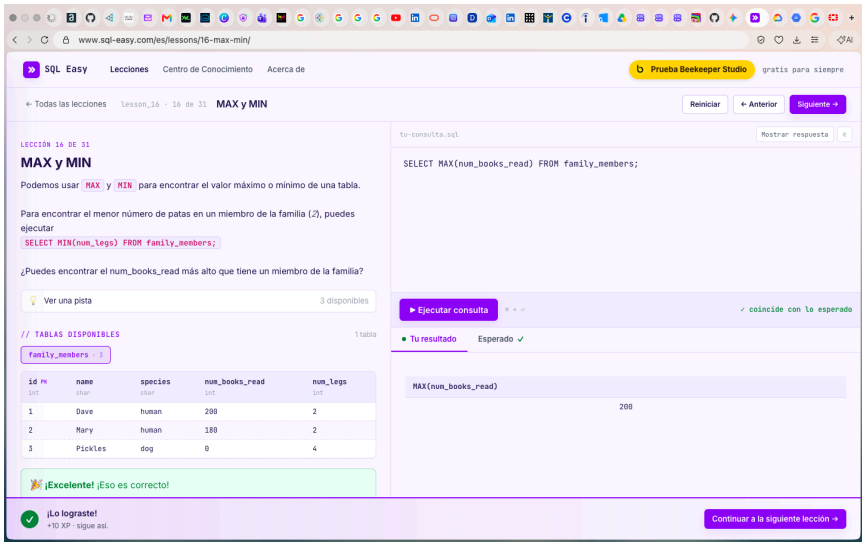
- **Objetivo del ejercicio:** Obtener la media aritmética o promedio de libros leídos por los miembros de la familia.
- **Concepto SQL:** Función de agregación `AVG()`
- **Consulta ejecutada:** `SELECT AVG(num_books_read) FROM family_members;`
- **Resultado:**



- **Interpretación:** Calcula el valor promedio dividiendo la suma total de libros leídos entre el número total de registros considerados.
- **Aprendizaje:** Aprendí a extraer tendencias estadísticas descriptivas directamente desde el motor de la base de datos.
- **Aplicación en el mundo real:** Aplicado en análisis de datos para conocer el salario promedio de una empresa, el tiempo promedio de respuesta de un servidor o la calificación media de un producto.

Ejercicio 16: Obtener valores máximos y mínimos con MAX y MIN

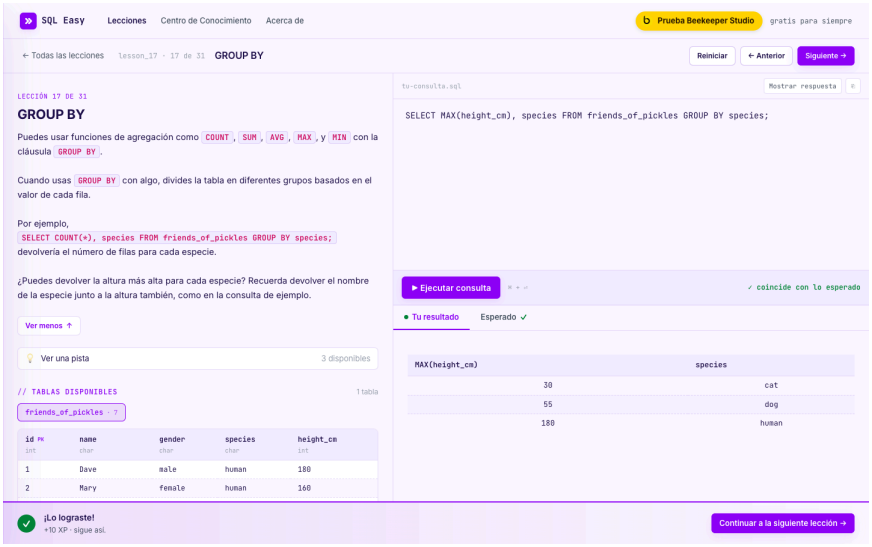
- **Objetivo del ejercicio:** Encontrar el número máximo de libros leídos dentro de la tabla `family_members`.
- **Concepto SQL:** Funciones de agregación `MAX()` y `MIN()`
- **Consulta ejecutada:** `SELECT MAX(num_books_read) FROM family_members;`
- **Resultado:**



- **Interpretación:** Analiza la columna numérica especificada y arroja el valor más alto registrado en ella.
- **Aprendizaje:** Comprendí la utilidad de las funciones de extremo para identificar rápidamente límites superiores o inferiores en los datos.
- **Aplicación en el mundo real:** Útil para identificar el récord de ventas de un empleado, el costo más alto de un proveedor o la temperatura máxima registrada en sensores IoT.

Ejercicio 17: Agrupación de datos con GROUP BY

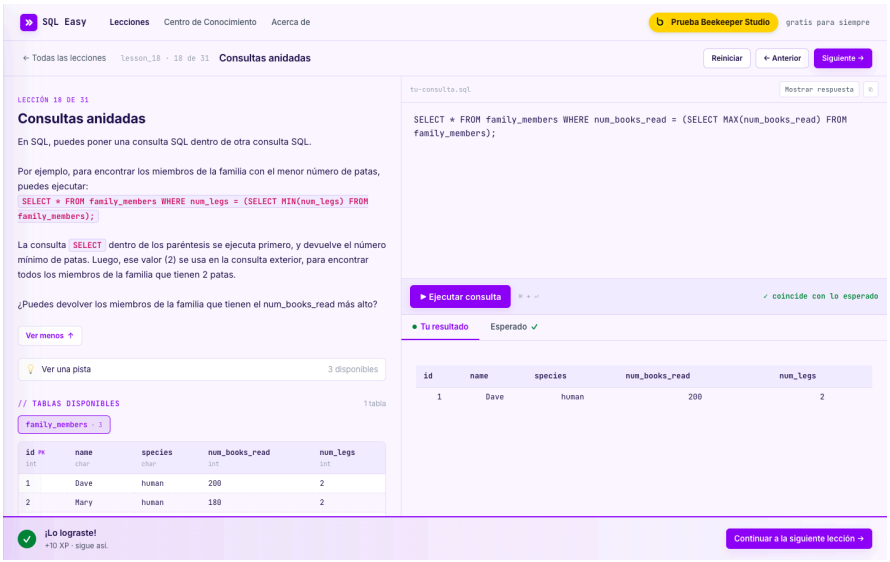
- **Objetivo del ejercicio:** Obtener la altura máxima registrada para cada una de las diferentes especies de amigos de Pickles.
- **Concepto SQL:** Cláusula `GROUP BY` combinada con `MAX()`
- **Consulta ejecutada:** `SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;`
- **Resultado:**



- **Interpretación:** Agrupa los registros por categoría de especie y calcula la altura máxima de manera independiente para cada grupo resultante.
- **Aprendizaje:** Entendí cómo estructurar consultas para realizar análisis segmentados por categorías, una de las herramientas más potentes en bases de datos.
- **Aplicación en el mundo real:** Indispensable para generar reportes por categorías de productos, departamentos de una empresa o sucursales geográficas.

Ejercicio 18: Consultas anidadas (Subconsultas)

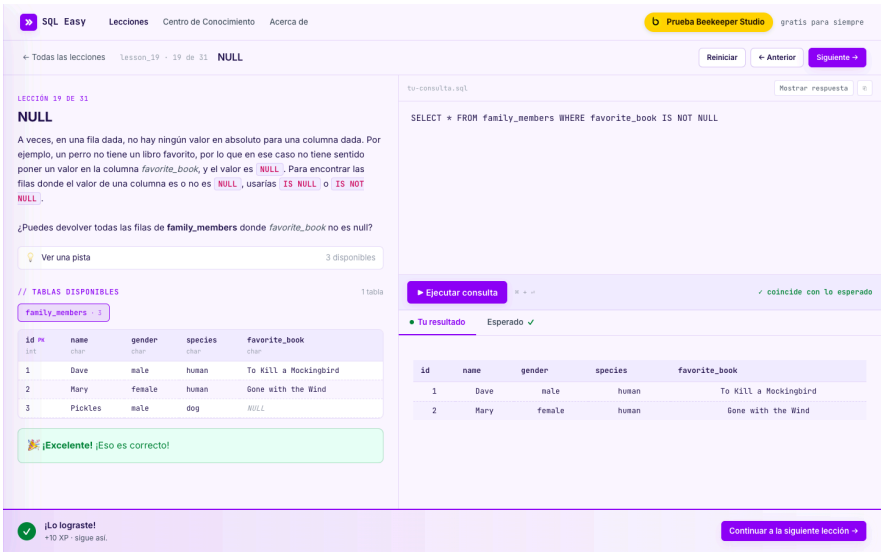
- **Objetivo del ejercicio:** Consultar los datos completos del miembro de la familia que haya leído la mayor cantidad de libros.
- **Concepto SQL:** Subconsulta anidada en el `WHERE`
- **Consulta ejecutada:** `SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);`
- **Resultado:**



- **Interpretación:** Primero calcula el valor máximo mediante una subconsulta interna y luego filtra la tabla principal para mostrar el registro que coincide exactamente con ese valor.
- **Aprendizaje:** Aprendí a resolver problemas complejos donde se requiere el resultado de una consulta previa para ejecutar el filtro principal.
- **Aplicación en el mundo real:** Utilizado para buscar transacciones que superen el promedio general de la empresa o localizar al cliente con el historial de compras más alto.

Ejercicio 19: Filtrado de nulos con IS NOT NULL

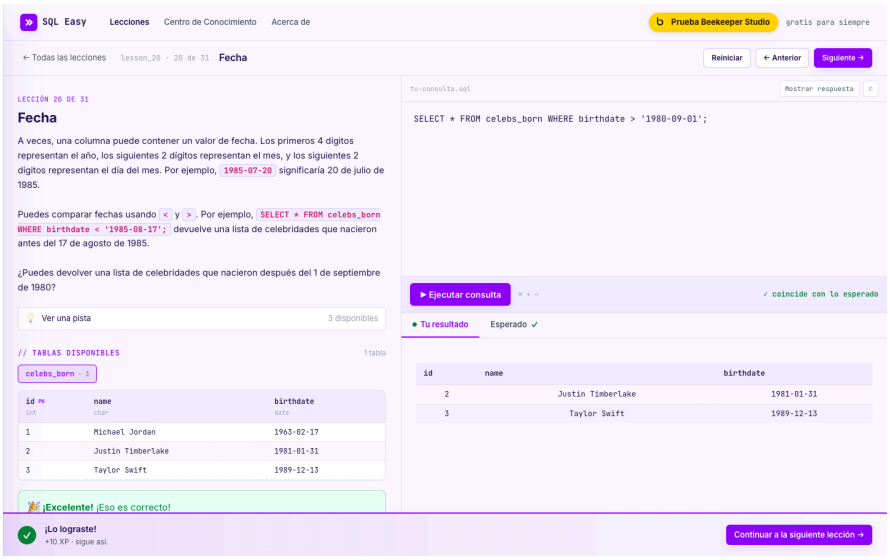
- **Objetivo del ejercicio:** Obtener los registros de la familia que sí tengan registrado un libro favorito (excluyendo valores vacíos).
- **Concepto SQL:** Operador de nulidad `IS NOT NULL`
- **Consulta ejecutada:** `SELECT * FROM family_members WHERE favorite_book IS NOT NULL;`
- **Resultado:**



- **Interpretación:** Filtra la tabla omitiendo todas aquellas filas donde el campo `favorite_book` no contenga información (sea nulo).
- **Aprendizaje:** Comprendí que los valores nulos (`NULL`) no se pueden evaluar con operadores comunes como `=` o `!=`, requiriendo operadores especiales de nulidad.
- **Aplicación en el mundo real:** Vital para verificar la integridad de los datos, por ejemplo, filtrar cuentas de usuario que sí tengan un correo electrónico o número de teléfono registrado.

Ejercicio 20: Filtrado por fechas

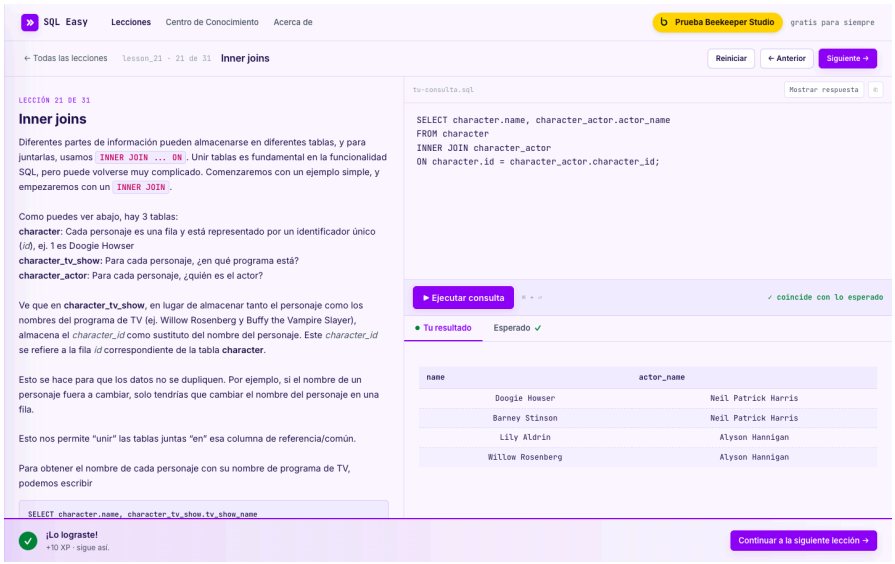
- **Objetivo del ejercicio:** Consultar los registros de celebridades cuya fecha de nacimiento sea posterior al 1 de septiembre de 1980.
- **Concepto SQL:** Comparación de tipos de datos de fecha (`DATE`)
- **Consulta ejecutada:** `SELECT * FROM celebs_born WHERE birthdate > '1980-09-01';`
- **Resultado:**



- **Interpretación:** Compara el campo de fecha con el formato de cadena estándar y devuelve los registros que se ubican cronológicamente después del límite establecido.
- **Aprendizaje:** Aprendí cómo los motores de bases de datos interpretan y comparan formatos de fecha y hora bajo estándares lógicos.
- **Aplicación en el mundo real:** Muy común en sistemas de gestión para filtrar registros creados después de cierto periodo fiscal, contratos vigentes o eventos agendados.

Ejercicio 21: Inner Join básico

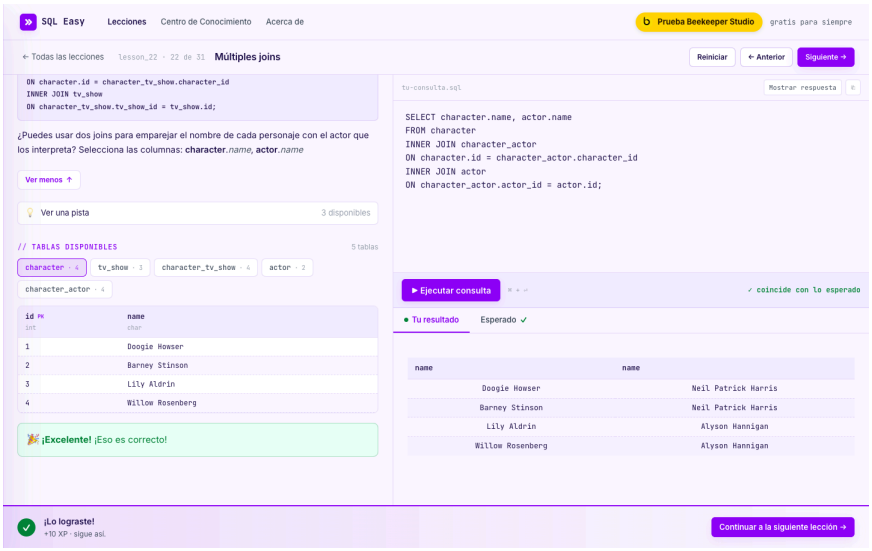
- **Objetivo del ejercicio:** Relacionar los nombres de los personajes con los nombres de sus respectivos actores utilizando una clave común.
- **Concepto SQL:** `INNER JOIN` y cláusula `ON`
- **Consulta ejecutada:** `SELECT character.name, character_actor.actor_name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id;`
- **Resultado:**



- **Interpretación:** Combina ambas tablas y devuelve únicamente las filas que tienen una coincidencia exacta de IDs en las dos tablas involucradas.
- **Aprendizaje:** Comprendí cómo fusionar datos de múltiples tablas relacionadas mediante llaves primarias y foráneas.
- **Aplicación en el mundo real:** Fundamental en sistemas relacionales para unir información de clientes con sus respectivas facturas o pedidos.

Ejercicio 22: Múltiples Joins

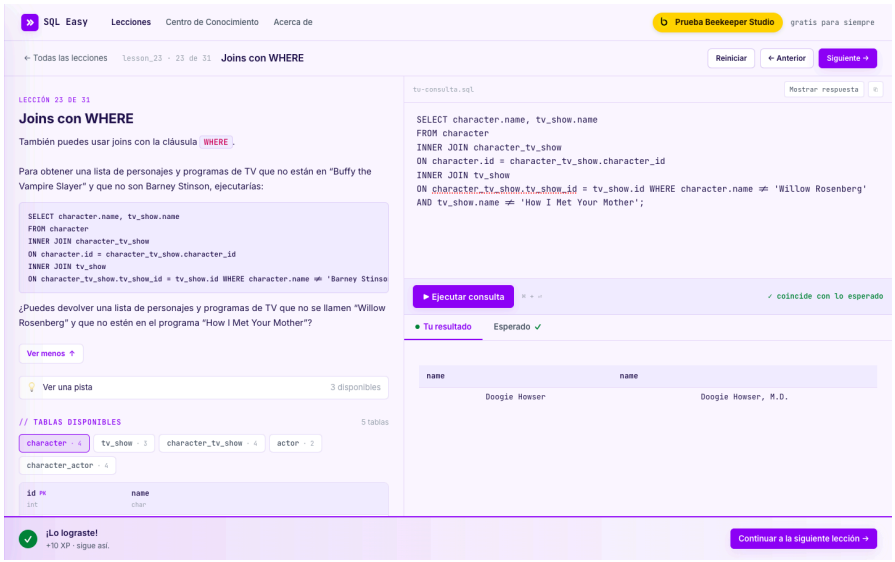
- **Objetivo del ejercicio:** Relacionar tres tablas distintas para obtener los nombres de los personajes junto con los nombres reales de los actores a través de una tabla intermedia.
- **Concepto SQL:** Múltiples `INNER JOIN` encadenados
- **Consulta ejecutada:** `SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id;`
- **Resultado:**



- **Interpretación:** Conecta tres tablas en cadena para cruzar datos complejos, resolviendo relaciones de muchos a muchos.
- **Aprendizaje:** Aprendí a estructurar consultas avanzadas que involucran más de dos tablas para extraer información normalizada de una base de datos relacional.
- **Aplicación en el mundo real:** Utilizado para consultar estructuras complejas, como qué productos compró un usuario en una sucursal específica a través de un sistema de ventas.

Ejercicio 23: Joins combinados con filtros WHERE

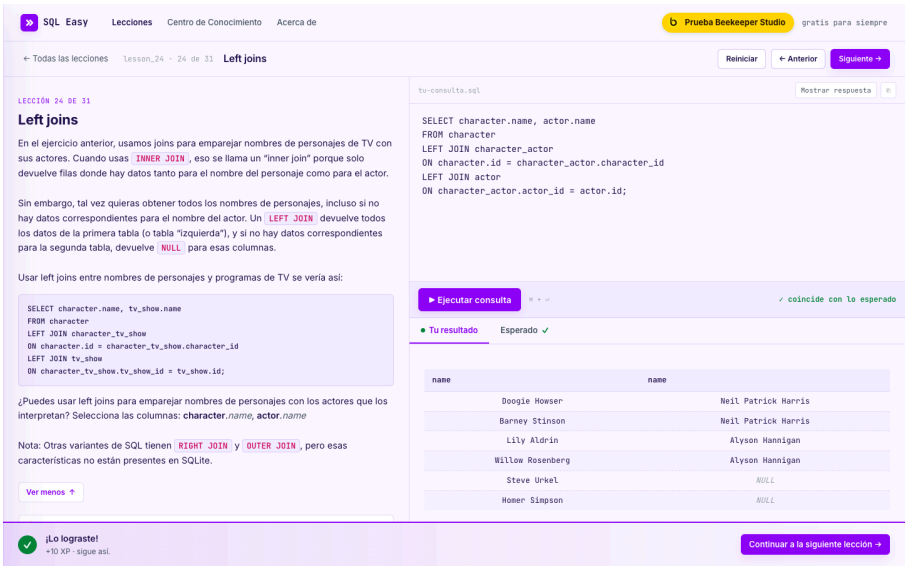
- **Objetivo del ejercicio:** Unir tablas de personajes y programas de televisión excluyendo registros específicos (como un personaje o un programa determinado).
- **Concepto SQL:** `INNER JOIN` múltiple combinado con filtros `WHERE` y operadores de desigualdad (`!=`)
- **Consulta ejecutada:** `SELECT character.name, tv_show.name FROM character INNER JOIN character_tv_show ON character.id = character_tv_show.character_id INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id WHERE character.name != 'Willow Rosenberg' AND tv_show.name != 'How I Met Your Mother';`
- **Resultado:**



- **Interpretación:** Realiza la unión de las tablas pero descarta del resultado final los registros que coincidan con las restricciones indicadas.
- **Aprendizaje:** Comprendí cómo aplicar filtros de exclusión sobre resultados obtenidos mediante uniones de tablas.
- **Aplicación en el mundo real:** Útil para filtrar reportes complejos de inventarios o usuarios, eliminando elementos inactivos o registros de prueba específicos.

Ejercicio 24: Left Joins

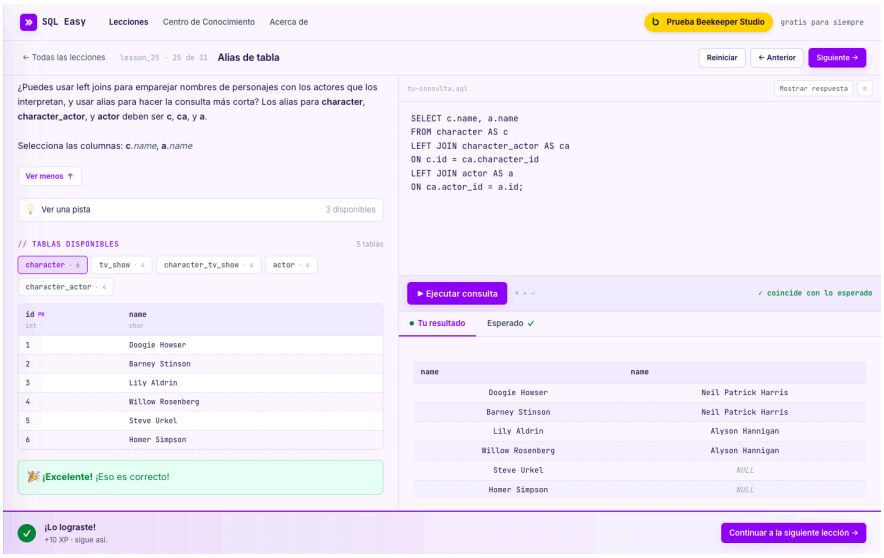
- **Objetivo del ejercicio:** Listar todos los personajes, mostrando el nombre del actor asociado si es que lo hay, sin excluir a los personajes que no tengan actores vinculados.
- **Concepto SQL:** `LEFT JOIN`
- **Consulta ejecutada:** `SELECT character.name, actor.name FROM character LEFT JOIN character_actor ON character.id = character_actor.character_id LEFT JOIN actor ON character_actor.actor_id = actor.id;`
- **Resultado:**



- **Interpretación:** Devuelve todos los registros de la tabla de la izquierda (personajes) y los empareja con la derecha; si no hay coincidencia, muestra valores nulos (`NULL`) en lugar de omitir la fila.
- **Aprendizaje:** Diferencié el `LEFT JOIN` del `INNER JOIN`, comprendiendo la importancia de conservar la integridad de la tabla principal aunque no existan datos cruzados.
- **Aplicación en el mundo real:** Indispensable para listar todos los clientes de una empresa, incluso aquellos que aún no han realizado ninguna compra.

Ejercicio 25: Alias de tabla

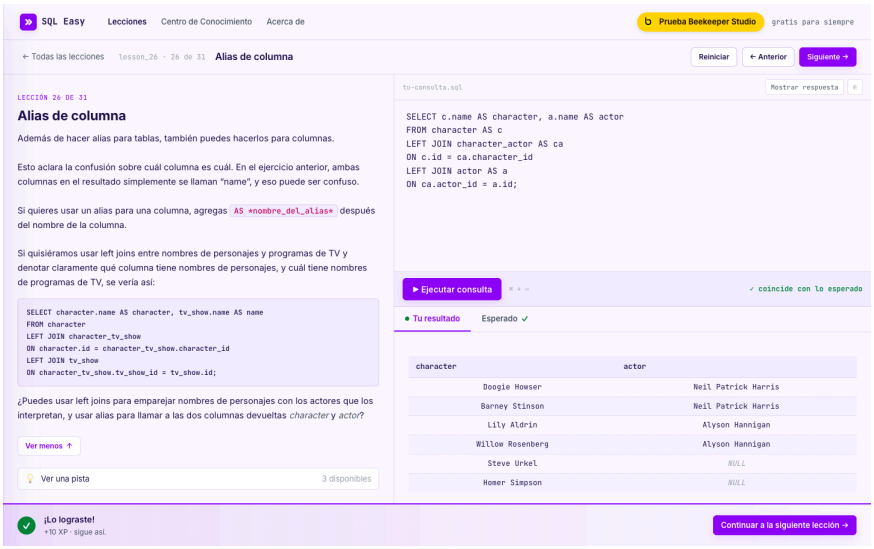
- **Objetivo del ejercicio:** Simplificar y optimizar la lectura de una consulta larga utilizando abreviaturas o alias para los nombres de las tablas.
- **Concepto SQL:** Alias de tablas con la cláusula `AS`
- **Consulta ejecutada:** `SELECT c.name, a.name FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;`
- **Resultado:**



- **Interpretación:** Asigna letras clave a cada tabla (`c`, `ca`, `a`) para referenciarlas rápidamente a lo largo de toda la consulta SQL.
- **Aprendizaje:** Aprendí a escribir código SQL más limpio, legible y fácil de mantener al evitar repetir nombres de tablas extensos.
- **Aplicación en el mundo real:** Práctica estándar en el desarrollo de software corporativo para optimizar consultas complejas en bases de datos empresariales.

Ejercicio 26: Alias de columna

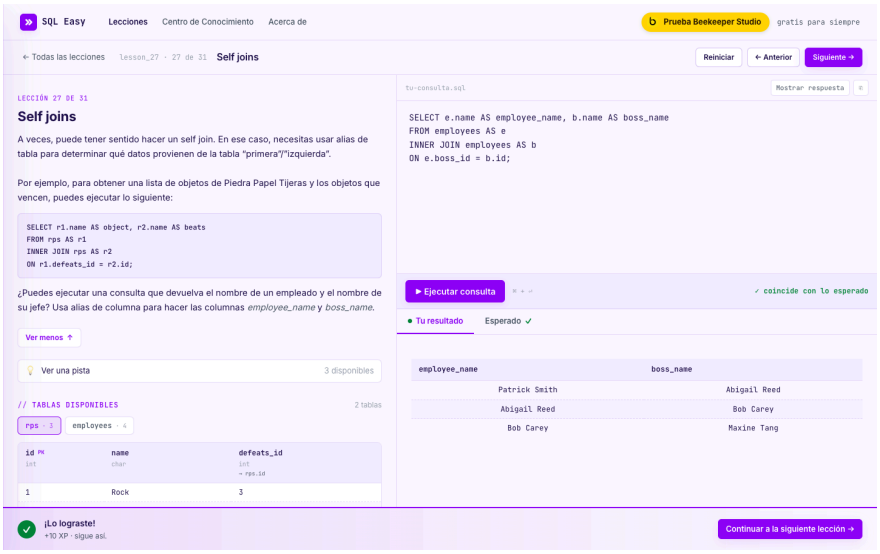
- **Objetivo del ejercicio:** Renombrar los encabezados de las columnas devueltas en el resultado de la consulta para que sean más descriptivos o limpios.
- **Concepto SQL:** Alias de columnas mediante `AS`
- **Consulta ejecutada:** `SELECT c.name AS character, a.name AS actor FROM character AS c LEFT JOIN character_actor AS ca ON c.id = ca.character_id LEFT JOIN actor AS a ON ca.actor_id = a.id;`
- **Resultado:**



- **Interpretación:** Modifica visualmente los nombres de los campos en la tabla de salida resultante (por ejemplo, renombrando columnas técnicas a etiquetas legibles como `character` y `actor`).
- **Aprendizaje:** Comprendí cómo personalizar la interfaz de presentación de los datos directamente desde el motor de base de datos.
- **Aplicación en el mundo real:** Utilizado frecuentemente para generar reportes ejecutivos donde los nombres de las columnas deben presentarse con espacios o etiquetas amigables para el usuario final.

Ejercicio 27: Self Joins

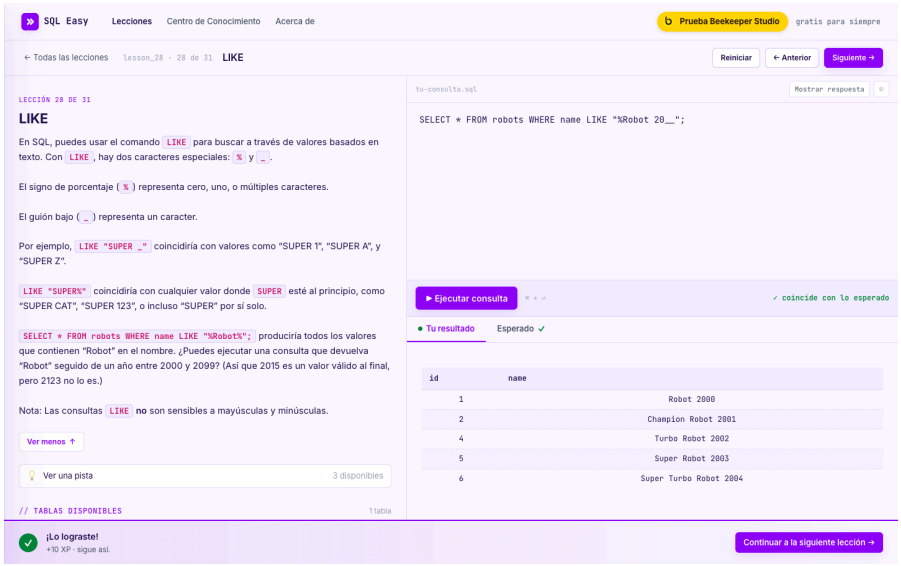
- **Objetivo del ejercicio:** Relacionar una tabla consigo misma para mostrar los nombres de los empleados junto con los nombres de sus respectivos jefes.
- **Concepto SQL:** `SELF JOIN` (Unión de una tabla con su propia instancia usando alias)
- **Consulta ejecutada:** `SELECT e.name AS employee_name, b.name AS boss_name FROM employees AS e INNER JOIN employees AS b ON e.boss_id = b.id;`
- **Resultado:**



- **Interpretación:** Trata a la misma tabla como si fueran dos tablas distintas mediante el uso de alias, permitiendo relacionar registros internos (empleado con supervisor).
- **Aprendizaje:** Aprendí a resolver estructuras jerárquicas o de árbol dentro de bases de datos relacionales planas.
- **Aplicación en el mundo real:** Muy útil en organigramas empresariales, sistemas de referidos o estructuras de categorías que dependen de subcategorías de la misma tabla.

Ejercicio 28: Patrones de búsqueda con LIKE

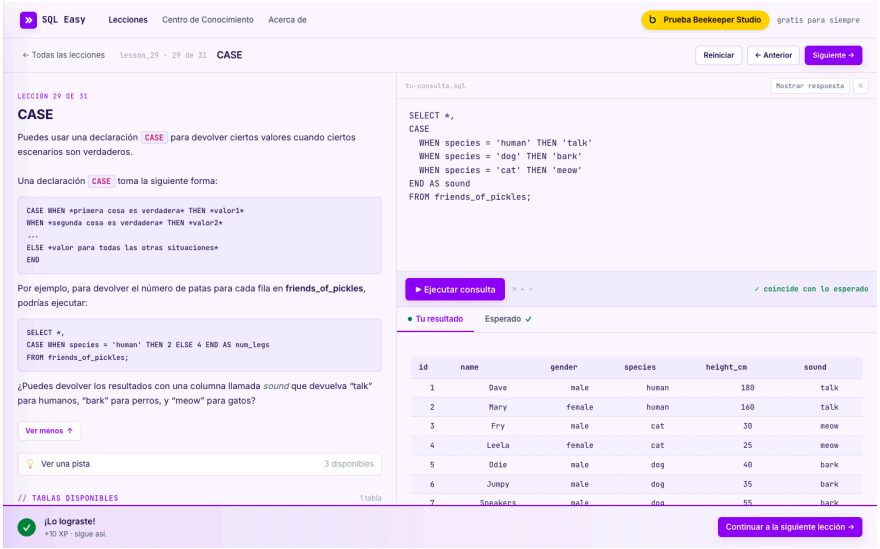
- **Objetivo del ejercicio:** Buscar robots cuyos nombres coincidan con un patrón específico utilizando comodines.
- **Concepto SQL:** Operador `LIKE` y comodines de texto
- **Consulta ejecutada:** `SELECT * FROM robots WHERE name LIKE "%Robot 20__";`
- **Resultado:**



- **Interpretación:** Filtra registros buscando coincidencias parciales de texto basadas en patrones estructurados con comodines.
- **Aprendizaje:** Comprendí cómo realizar búsquedas flexibles de texto cuando no se conoce el valor exacto de un registro.
- **Aplicación en el mundo real:** Esencial en barras de búsqueda de aplicaciones web, para encontrar usuarios por nombres parciales o correos electrónicos.

Ejercicio 29: Condicionales lógicas con CASE

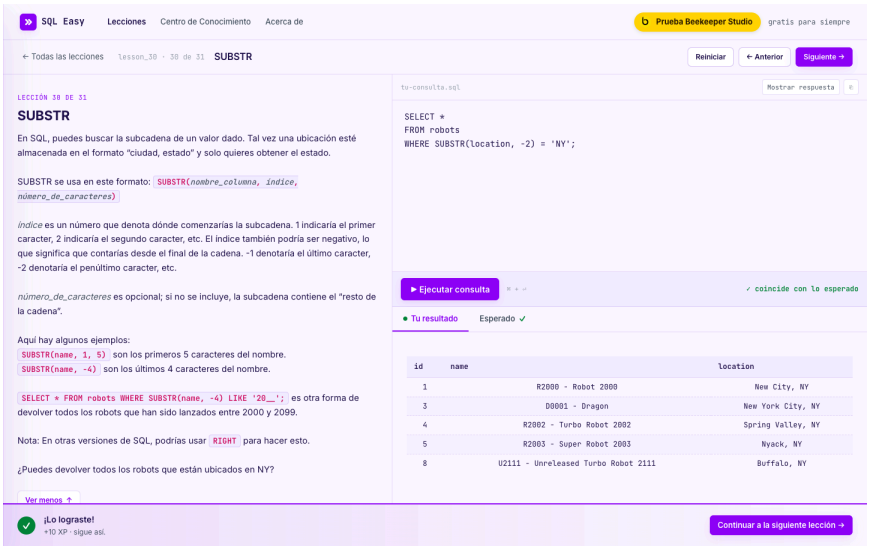
- **Objetivo del ejercicio:** Crear una columna dinámica calculada que asigne un sonido o etiqueta diferente dependiendo de la especie de cada registro.
- **Concepto SQL:** Estructura condicional `CASE ... WHEN ... THEN ... END`
- **Consulta ejecutada:** `SELECT *, CASE WHEN species = 'human' THEN 'talk' WHEN species = 'dog' THEN 'bark' WHEN species = 'cat' THEN 'meow' END AS sound FROM friends_of_pickles;`
- **Resultado:**



- **Interpretación:** Evalúa condiciones fila por fila de manera lógica para generar un valor personalizado en una nueva columna virtual.
- **Aprendizaje:** Aprendí a implementar lógica de programación condicional directamente en las consultas de SQL sin alterar la estructura física de la base de datos.
- **Aplicación en el mundo real:** Muy útil para transformar códigos numéricos de estatus en descripciones legibles (ej. cambiar 1 por "Activo" y 0 por "Inactivo") en reportes.

Ejercicio 30: Manipulación de cadenas con SUBSTR

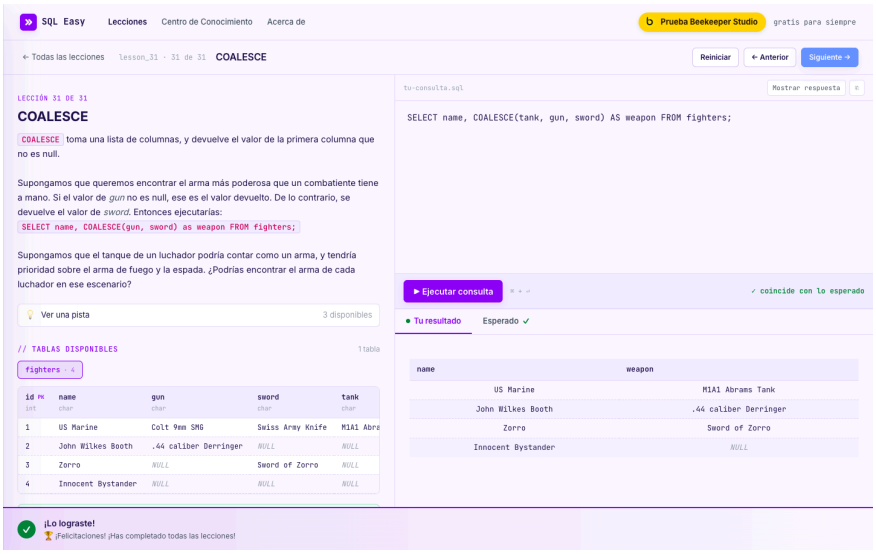
- **Objetivo del ejercicio:** Filtrar robots cuya ubicación finalice con una combinación de caracteres específica (ej. `'NY'`).
- **Concepto SQL:** Función de manipulación de texto `SUBSTR()`
- **Consulta ejecutada:** `SELECT * FROM robots WHERE SUBSTR(location, -2) = 'NY';`
- **Resultado:**



- **Interpretación:** Extrae una porción específica de una cadena de texto (empezando desde el final con índices negativos) y aplica el filtro de búsqueda sobre ese fragmento.
- **Aprendizaje:** Comprendí el uso de funciones de texto avanzadas para recortar y analizar partes específicas de los datos almacenados.
- **Aplicación en el mundo real:** Útil para validar códigos postales, extraer extensiones de archivos o filtrar prefijos telefónicos y regiones geográficas.

Ejercicio 31: Manejo de valores alternativos con COALESCE

- **Objetivo del ejercicio:** Mostrar el primer valor disponible entre varias columnas opcionales de armas para cada luchador.
- **Concepto SQL:** Función de manejo de nulos `COALESCE()`
- **Consulta ejecutada:** `SELECT name, COALESCE(tank, gun, sword) AS weapon FROM fighters;`
- **Resultado**



- **Interpretación:** Evalúa una lista de columnas en orden y devuelve el primer valor que no sea nulo (`NULL`), evitando mostrar campos vacíos.
- **Aprendizaje:** Aprendí a gestionar la ausencia de datos seleccionando alternativas en cascada dentro de una consulta.
- **Aplicación en el mundo real:** Muy utilizado para mostrar información de contacto alternativa (ej. mostrar el correo secundario si el principal no está disponible).

REFLEXIONA Y CONCLUYE (SQL EASY)

Las consultas de selección básica, filtros sencillos, operadores de comparación y delimitación de resultados, ya que responden a estructuras lógicas directas. La construcción y comprensión de uniones complejas (`JOIN`, Múltiples Joins y Self Joins) y consultas anidadas, donde se requiere mapear las relaciones entre llaves y múltiples tablas simultáneamente. En entornos corporativos y de ciberseguridad, la habilidad para estructurar consultas relacionales complejas permite auditar logs de accesos cruzados, analizar transacciones financieras sospechosas, cruzar bases de datos de clientes e inventarios, y realizar análisis forenses sobre tablas de datos interconectadas.
La práctica intensiva con los 31 ejercicios de SQL Easy fortaleció las bases lógicas indispensables para interactuar con bases de datos relacionales, permitiendo no solo extraer datos con precisión sino también comprender la arquitectura subyacente que las aplicaciones web utilizan, aspecto clave para la evaluación y aseguramiento del backend en ciberseguridad.